Infosys | Springboard

⇲PROJECT DOCUMENTATION

TrafficVision AI

~Smart Traffic Prediction & Congestion Management System

Project Guide

- Ms.Shravya U

Team Members

1. Archee Sinha
2. Srilekha K
3. Chandini Korukonda
4. Gaurav Bawankar

2) ABSTRACT

TrafficVision AI is an intelligent traffic management platform designed to monitor traffic conditions, predict congestion, recommend alternate routes, generate traffic alerts, and provide analytical insights through a web-based application. The system combines a React.js frontend with a FastAPI backend and uses SQLite with SQLAlchemy for data management. Traffic information such as traffic volume, average speed, time-related features, weather conditions, road type, city zone, and accident status is processed for congestion prediction.

A Random Forest classification model is used to classify traffic conditions into Low, Medium, and High congestion levels. The trained model is integrated into the backend to provide prediction results along with confidence values. The system also connects live traffic information with AI-based recommendations to assist traffic operators in identifying congestion and deciding whether to monitor, reroute, or take no action. Interactive maps using React Leaflet and OpenStreetMap provide visual traffic monitoring, while analytics, alerts, and reports support better decision-making.

The application implements role-based access for administrators, traffic operators, and public users. The complete system is organized using Git and GitHub for version control and can be containerized using Docker and Docker Compose. TrafficVision AI demonstrates how AI-based prediction, real-time traffic monitoring, route recommendation, and analytics can be combined into a unified smart traffic management solution.

3) INTRODUCTION

3.1 Background

Rapid urbanization has significantly increased the number of vehicles on city roads, while road infrastructure has not scaled at the same pace. Traffic authorities and transportation agencies increasingly rely on technology — sensors, cameras, GPS data, and predictive analytics — to monitor and manage traffic more effectively. Smart-city initiatives around the world are adopting AI-driven traffic systems to reduce congestion, cut commute times, and support faster emergency response.

3.2 Problem Statement

Most existing traffic-monitoring setups are reactive rather than predictive — they show what is happening right now but give little advance warning of where and when congestion will build up. This makes it difficult for traffic authorities to intervene early, for commuters to plan around congestion, and for cities to make data-driven decisions about road usage. There is a need for a platform that combines real-time monitoring with AI-based forecasting and actionable route/alert recommendations.

3.3 Objectives

- Continuously monitor live traffic conditions (vehicle density, road utilization) across monitored road segments.
- Predict traffic congestion levels using AI/ML models based on relevant traffic features.
- Recommend alternate routes based on congestion levels and road conditions
- Generate timely alerts for congestion, accidents, and road closures.
- Provide analytics dashboards with traffic trends, heatmaps, and historical insights for decision-making.
- Support role-based access for traffic authorities, traffic operators, and the public.

4) SCOPE

4.1 In Scope

•User authentication and role-based access control (Admin/Authority, Operator, Public).
•Live traffic monitoring dashboard with vehicle density and road utilization tracking.
•AI/ML-based congestion prediction.
•Route analysis with alternate-route suggestions and travel-time estimation.
•Alert and notification workflows for congestion, accidents, and road closures.
•Analytics dashboard with traffic trend reports, congestion heatmaps, and historical insights.
•Deploy the platform using Docker and Docker Compose for containerized application deployment..

4.2 Out Of Scope

• Direct control or actuation of physical traffic signals / infrastructure.
• Integration with live government traffic-camera networks (simulated/sample data sources are used instead).
• Native mobile applications (the platform is delivered as a responsive web application).
• City-wide, multi-region deployment at full production scale.

5) EXISTING SYSTEM & PROPOSED SOLUTION

5.1 EXISTING SYSTEM

Most existing traffic-management systems primarily focus on monitoring current traffic conditions through maps, CCTV monitoring, and traffic applications. These systems provide information about present traffic conditions, but may have limited AI-based congestion prediction and integrated decision-support features. Route suggestions and alerts may also depend mainly on current traffic information, making it difficult for users and traffic operators to make proactive traffic-management decisions.

5.2 PROPOSED SOLUTION

TrafficVision AI provides a centralized web-based platform that combines traffic monitoring, AI-based congestion prediction, alternate route recommendation, alerts, analytics, and reports. The system processes traffic-related features such as traffic volume, average speed, time, city zone, road type, weather, and accident status to predict congestion levels. The prediction results are integrated with traffic information to provide recommendations such as reroute, monitor, or no action, helping traffic operators and users make better traffic-management decisions.

5.3 KEY FEATURES

- Role-based dashboard for authorities, operators, and the public.
- Real-time vehicle density and congestion monitoring.
- AI-based congestion and peak-hour prediction.
- Alternate-route recommendation with travel-time estimation.
- Automated alerts for congestion, accidents, and road closures.
- Analytics dashboard with heatmaps and historical traffic trends.

6) TECHNOLOGY STACK

1. **Programming Languages:** Python, JavaScript
2. **Frontend:** React.js, HTML, CSS
3. **Backend:** Python, FastAPI (REST APIs)
4. **Database:** SQLite, SQLAlchemy
5. **AI/ML:** Scikit-learn, Random Forest
6. **Maps & APIs:** React Leaflet, OpenStreetMap, REST APIs, JSON
7. **Deployment & Tools:** Docker, Docker Compose, Uvicorn, Nginx, Git, GitHub, Swagger UI

7) SYSTEM ARCHITECTURE

TrafficVision AI follows a client-server architecture consisting of a React.js frontend, FastAPI backend, SQLite database, AI/ML prediction component, and map-based traffic visualization.

7.1 System Modules

1. User Management Module

• User authentication, role-based access control, profile management, and role-specific access.

2. Traffic Monitoring Module

• Live traffic monitoring, road-wise traffic conditions, congestion-level visualization, and traffic dashboard.

3. Traffic Prediction Module

• AI-based congestion prediction using traffic volume, average speed, time, city zone, road type, weather, and accident status.

4. Route Analysis Module

• Alternate route recommendation based on congestion levels and road conditions.

5. Alert & Notification Module

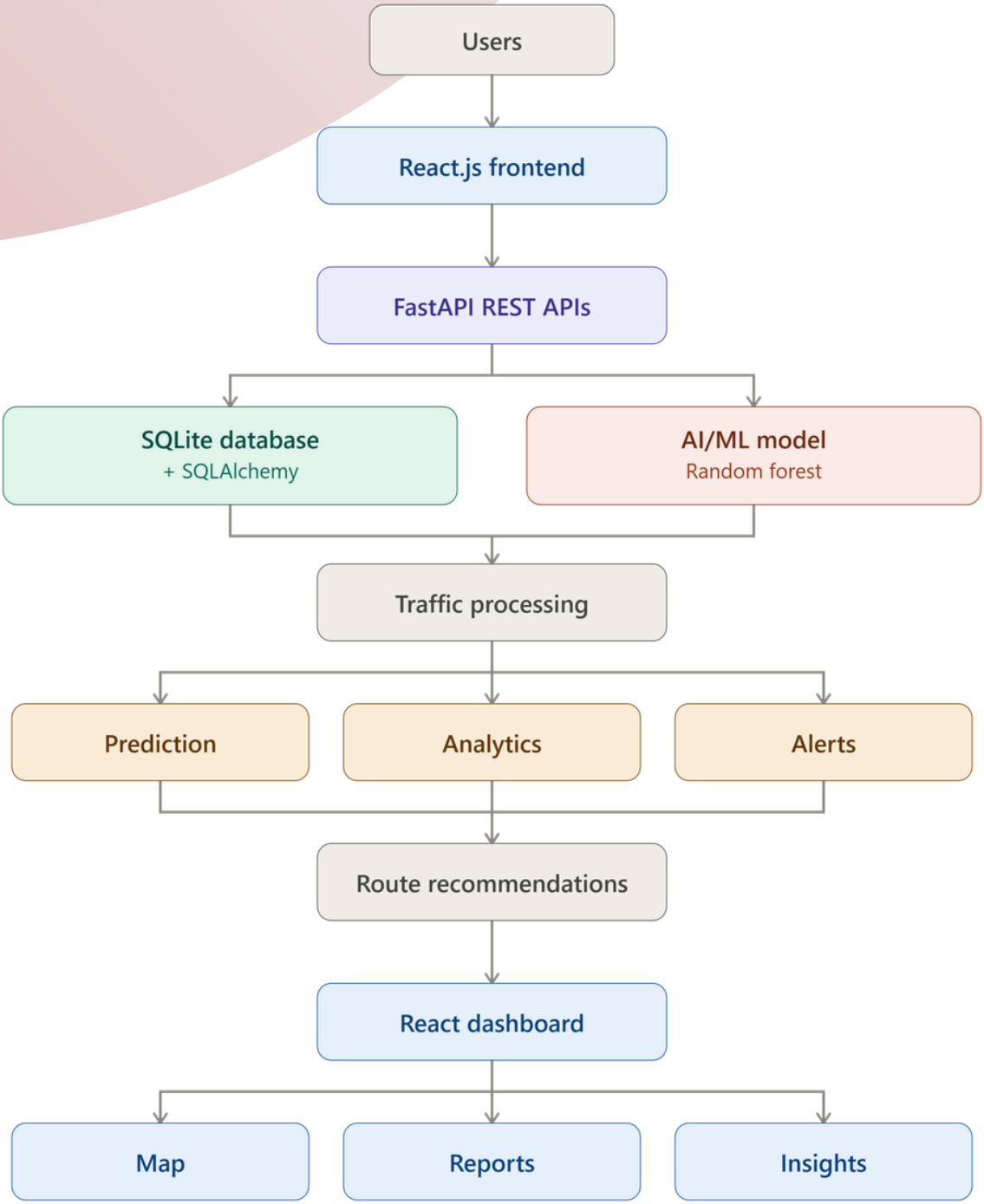
• Traffic congestion alerts, accident-related alerts, and traffic condition notifications.

6. Analytics Dashboard Module

• Traffic analytics, congestion visualization, traffic trends, road performance insights, and historical traffic analysis.

7. Reports Module

• Traffic summary reports, AI recommendation reports, historical traffic reports, and CSV/PDF report export.


Users
React.js frontend
FastAPI REST APIs
SQLite database
+ SQLAlchemy
AI/ML model
Random forest
Traffic processing
Prediction
Analytics
Alerts
Route recommendations
React dashboard
Map
Reports
Insights

8) AI/ML IMPLEMENTATION

8.1 Dataset

TrafficVision AI uses traffic data containing information required for congestion prediction. The dataset includes traffic volume, average speed, hour of the day, day of the week, city zone, road type, weather condition, and accident status.
These features are used to identify traffic patterns and classify the congestion level into Low, Medium, and High categories. The traffic data is also used by the backend for monitoring, prediction, analytics, and AI-based recommendations.

8.2 Data Processing

- Relevant features are selected based on their importance to traffic congestion.
- Numerical features such as traffic volume, average speed, and time-related values are used directly.
- Categorical features such as city zone, road type, weather condition, and accident status are encoded into numerical values.
- The dataset is divided into 80% training data and 20% testing data.
- The same feature encoding is maintained during prediction to ensure consistency between training and the deployed model.

8.3 Model / Algorithm

- TrafficVision AI uses the Random Forest Classifier from Scikit-learn for traffic congestion classification.
- The model uses the selected traffic features to predict one of three congestion levels:Low,Medium,High
- A Random Forest model with 200 decision trees (n_estimators = 200) is used. Random Forest was selected because it works effectively with multiple traffic-related features and can handle both numerical and encoded categorical inputs.
- The trained model is saved as a model bundle and integrated into the FastAPI backend. The prediction API processes the input features and returns the predicted congestion level along with its confidence value.

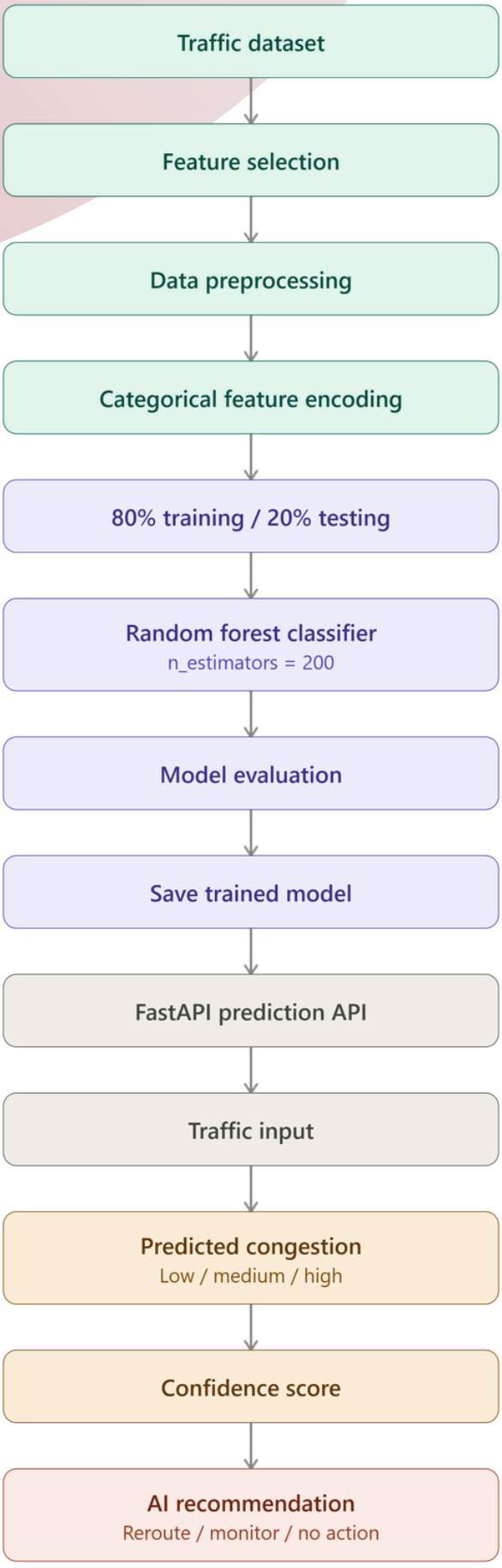
8.4 Training & Evaluation

- The prepared dataset is divided into 80% training data and 20% testing data. The Random Forest Classifier is trained using the selected traffic features and then evaluated using the test data.

- The model was tested with different traffic conditions to verify whether it correctly identifies Low, Medium, and High congestion levels. The trained model was then integrated with the backend prediction API and tested through the application.

- The model is also connected with traffic data to support AI-based traffic recommendations. Based on the predicted congestion condition, the system can provide recommendations such as reroute, monitor, or no action.

8.5 AI/ML WORKFLOW



9)DATABASE & API

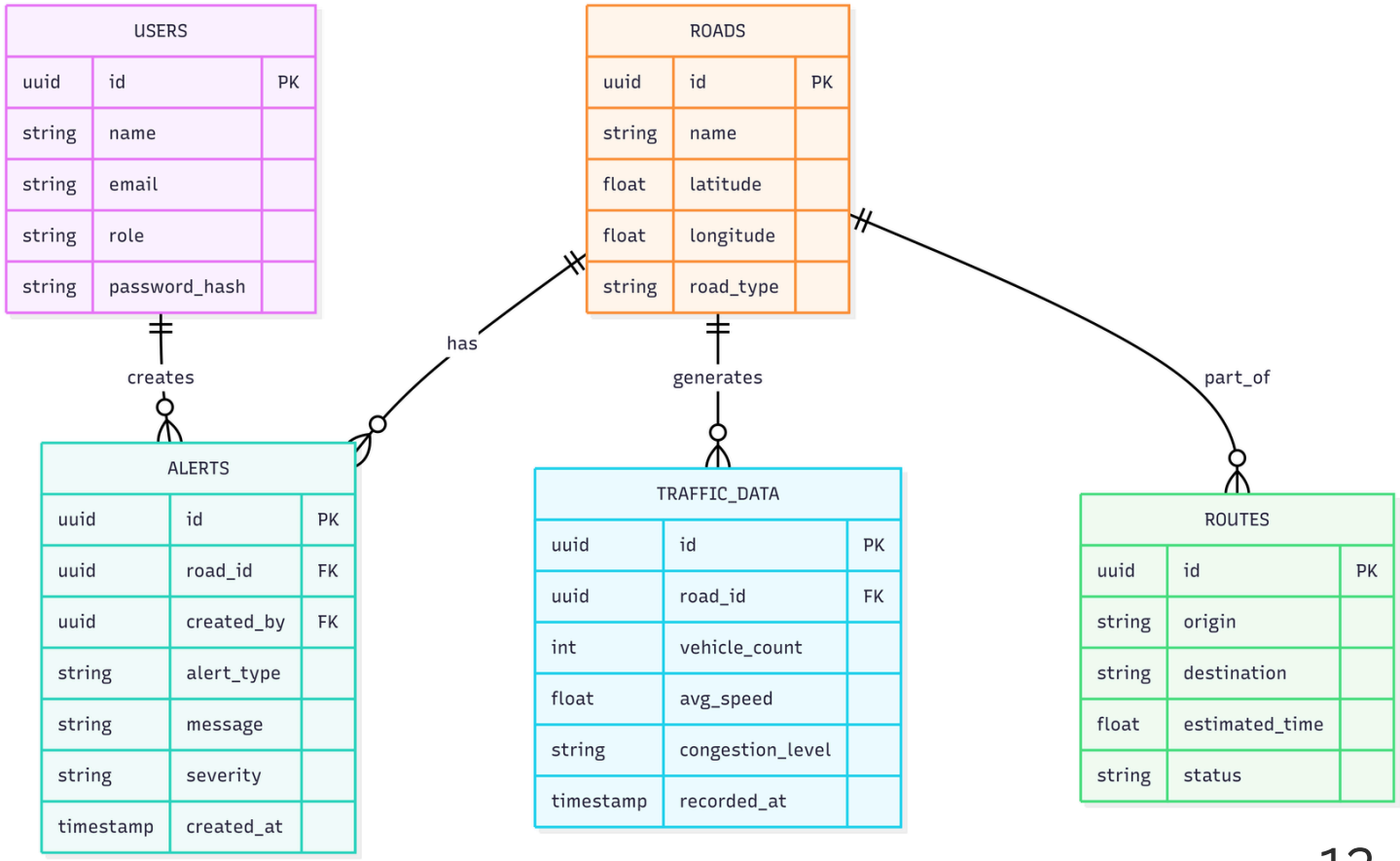
9.1 Database

The backend uses SQLAlchemy for database interaction. Traffic data is stored in a relational structure and provides information required for monitoring, prediction, route recommendation, analytics, and reports. The development project includes a SQLite database.

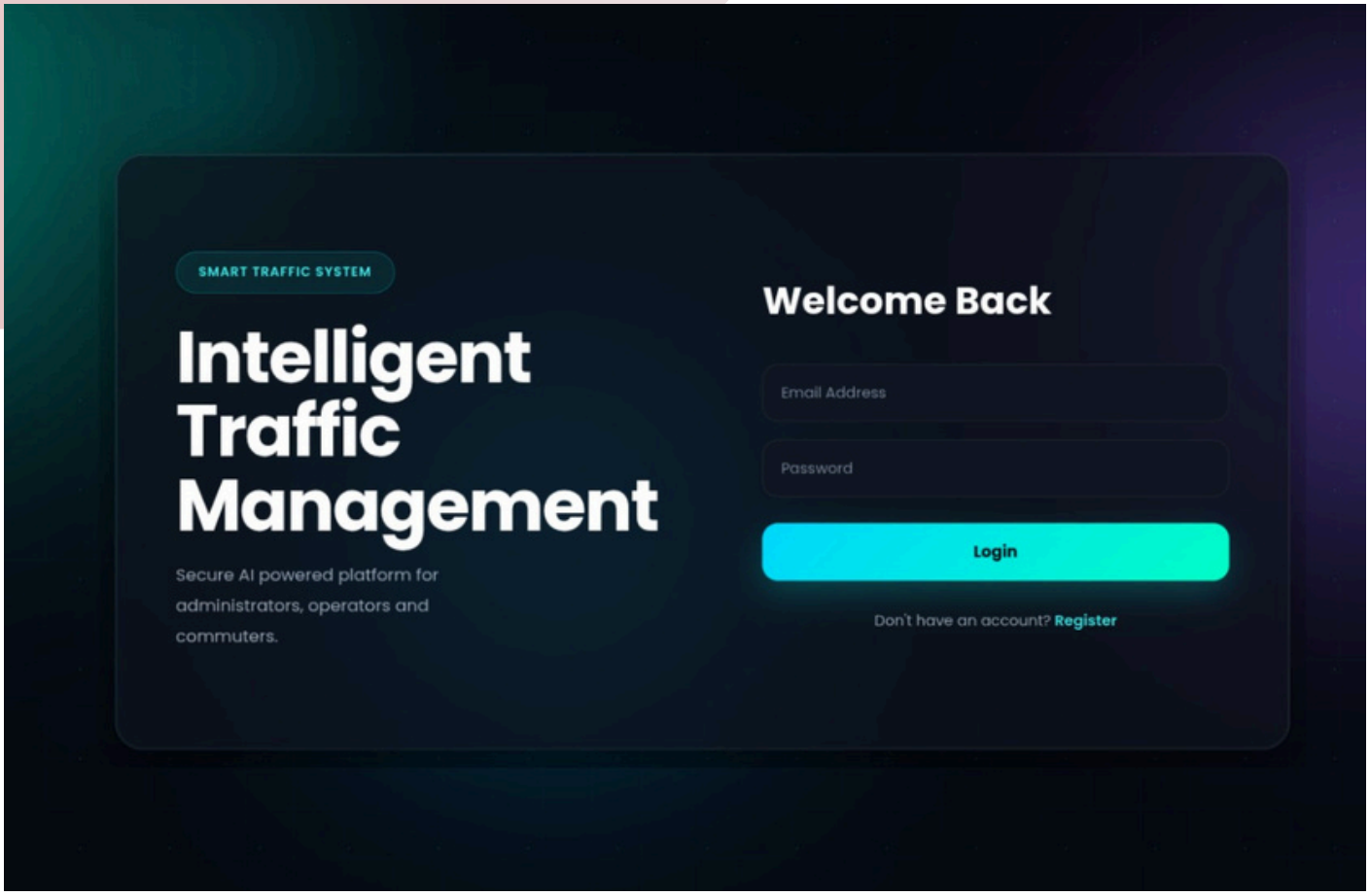
Traffic information includes road name, traffic volume, average speed, time-related attributes, weather/context information, accident information, and congestion level. Application entities also support authentication and role-based access.

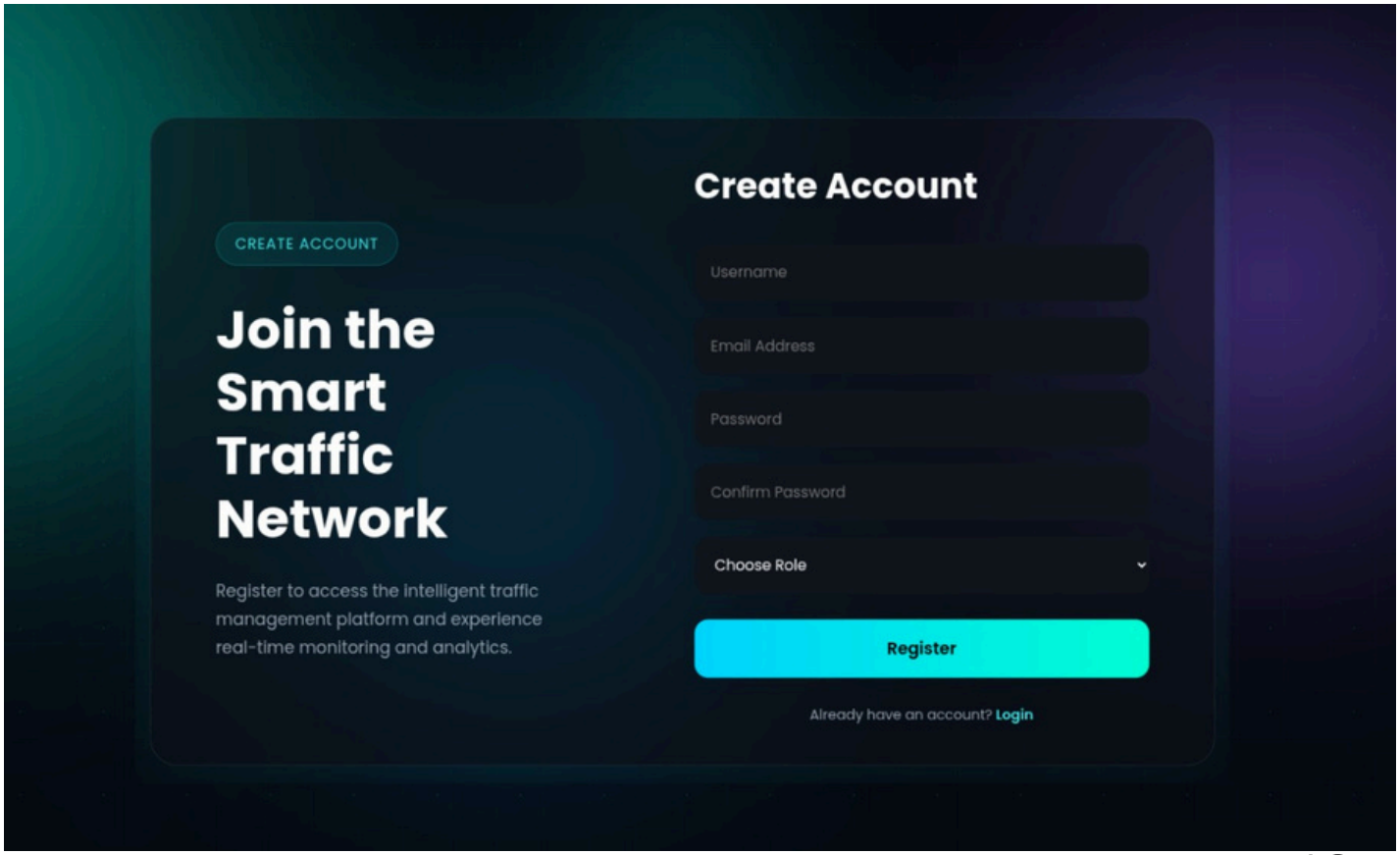
9.2 Data Processing

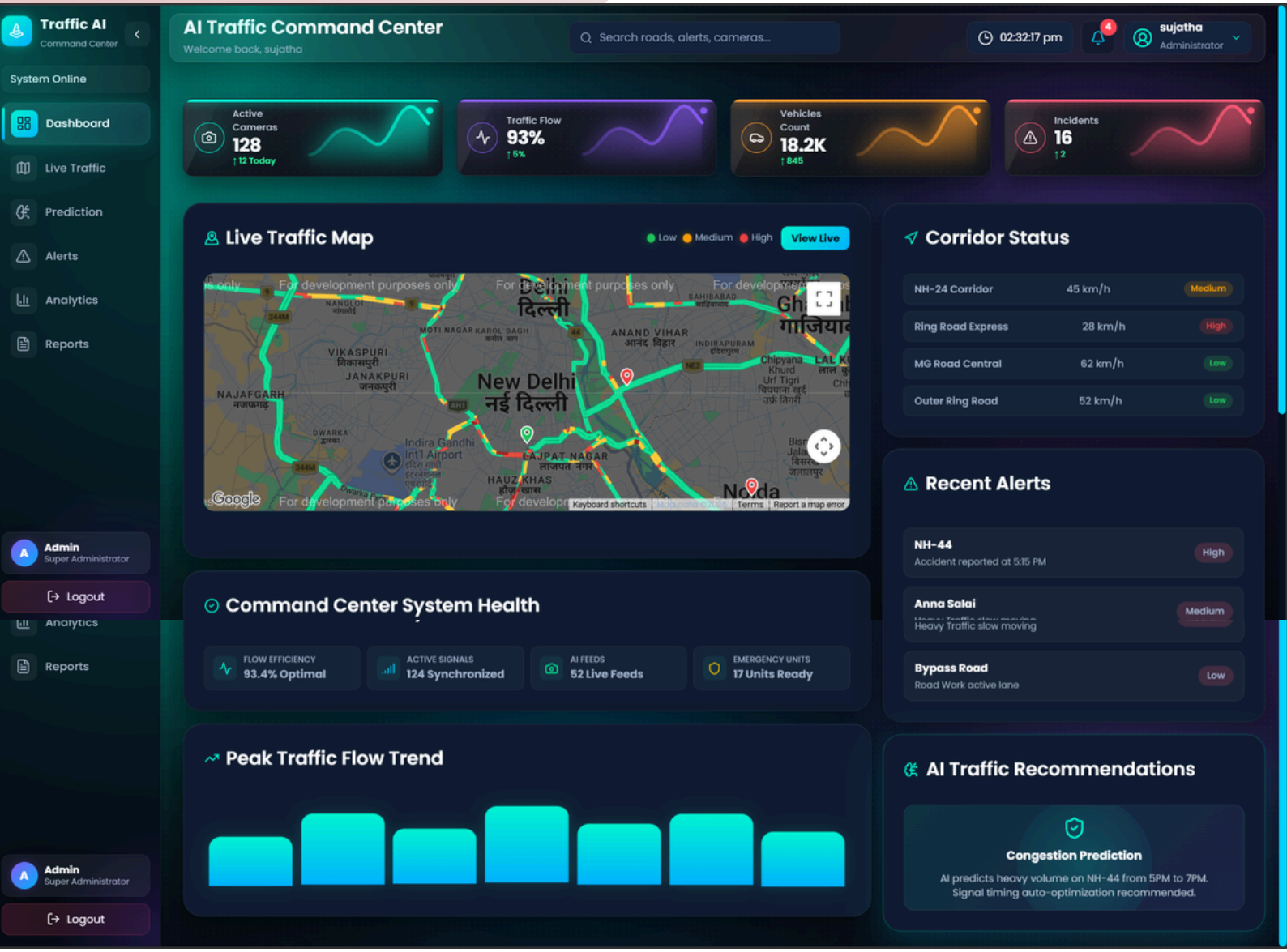
The backend exposes a REST API consumed by the frontend, broadly grouped by module: /auth (login, registration, token refresh), /traffic (live readings, road status), /predictions (congestion prediction), /routes (alternate-route and travel-time queries), /alerts (create/list/acknowledge alerts), and /analytics (trend, heatmap, and historical-report endpoints)./reports — traffic summary reports, AI recommendations, historical reports, and CSV/PDF export .All endpoints other than /auth require a valid JWT and are authorized according to the caller's role.



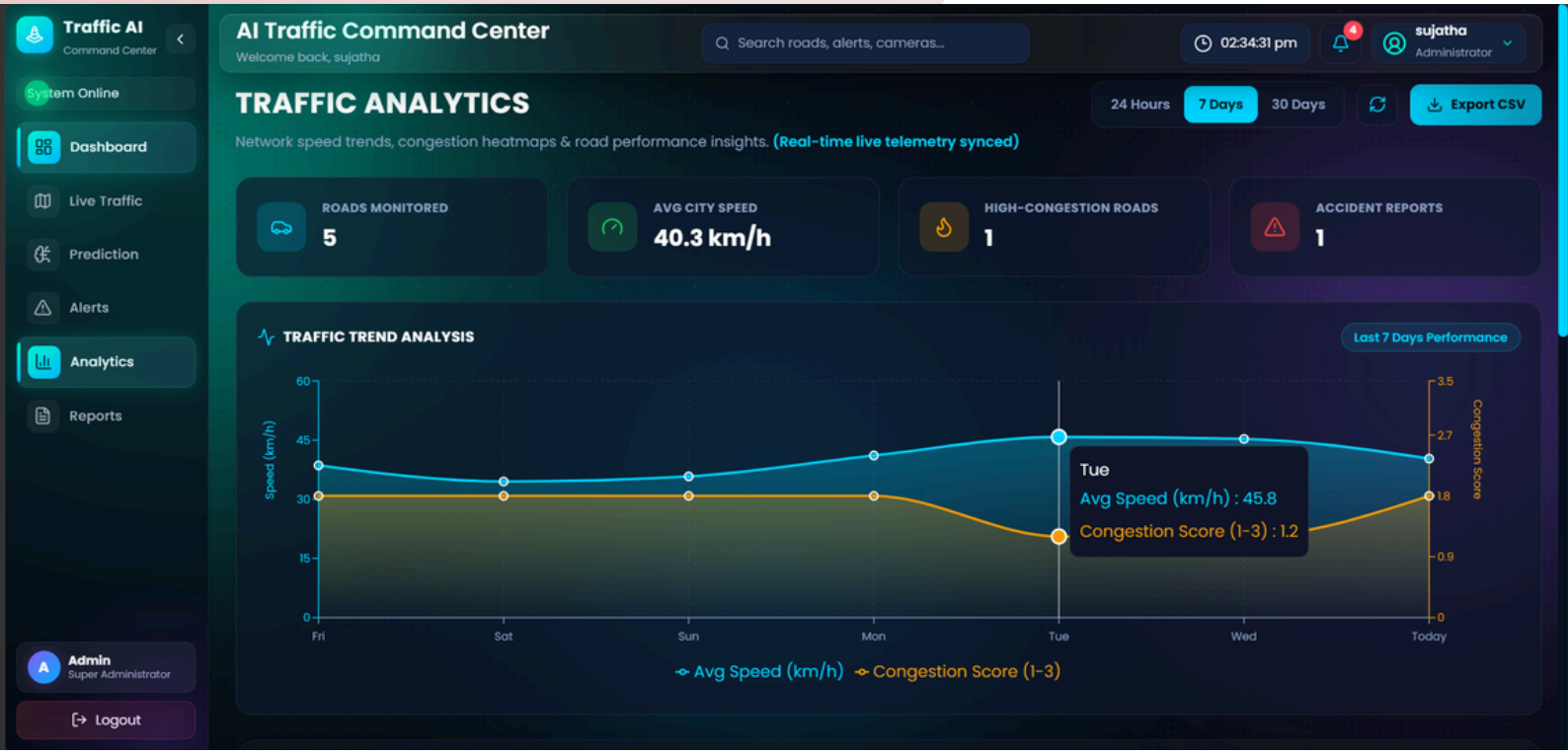
10)USER INTERFACE



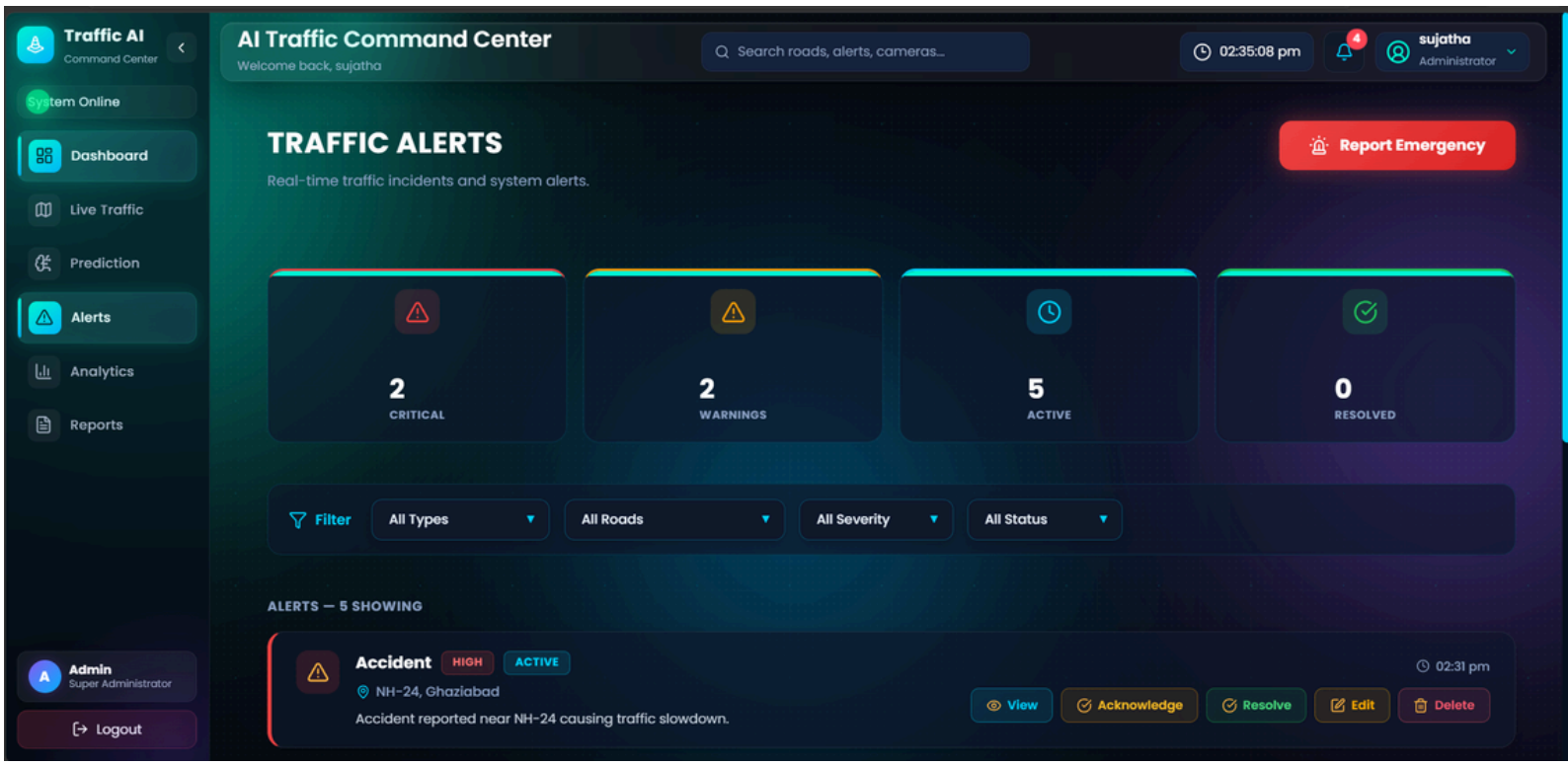




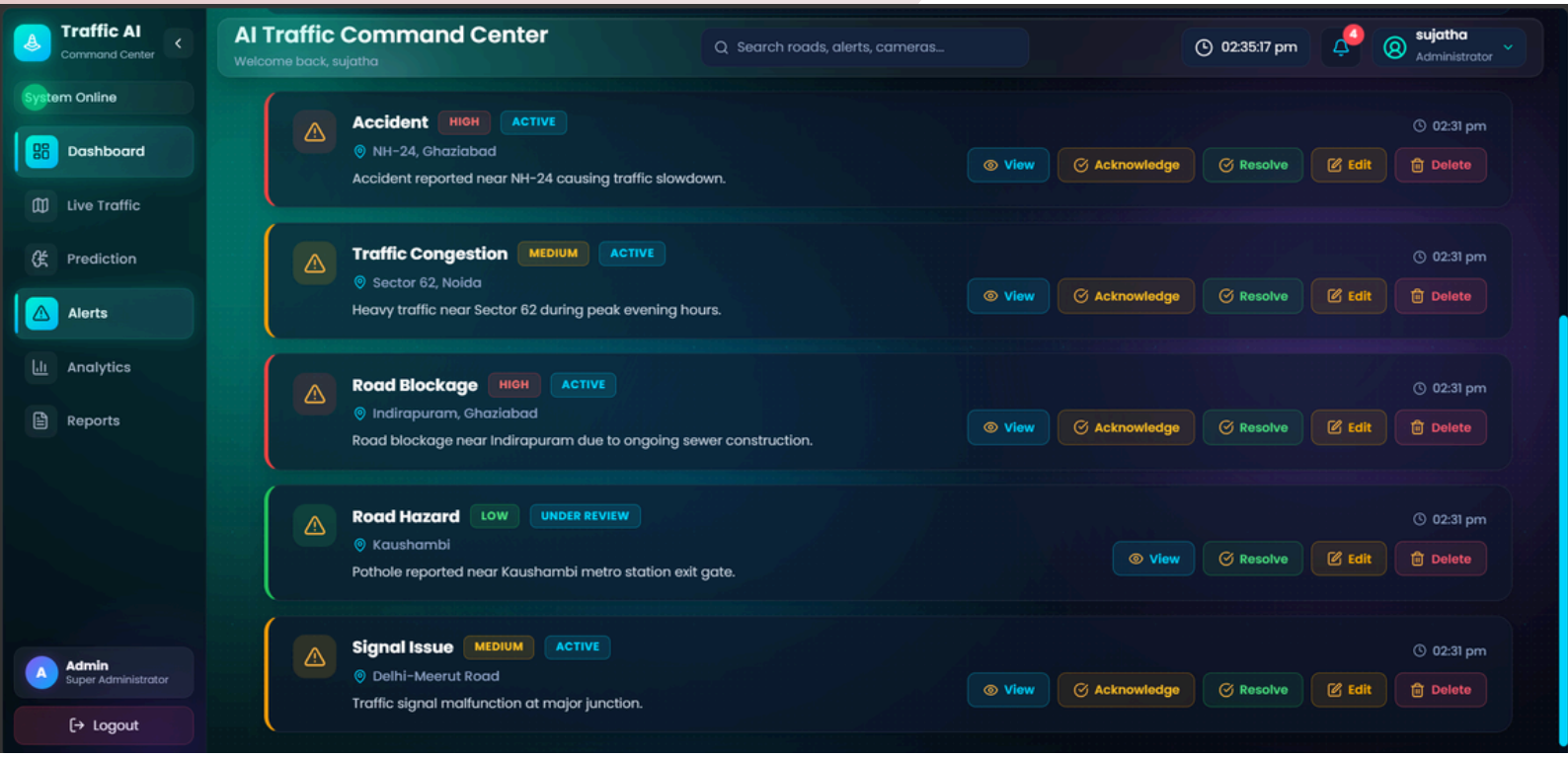
Dashboard: Displays the overall traffic monitoring status, including live traffic maps, road conditions, recent alerts, system health, and AI-based traffic recommendations.

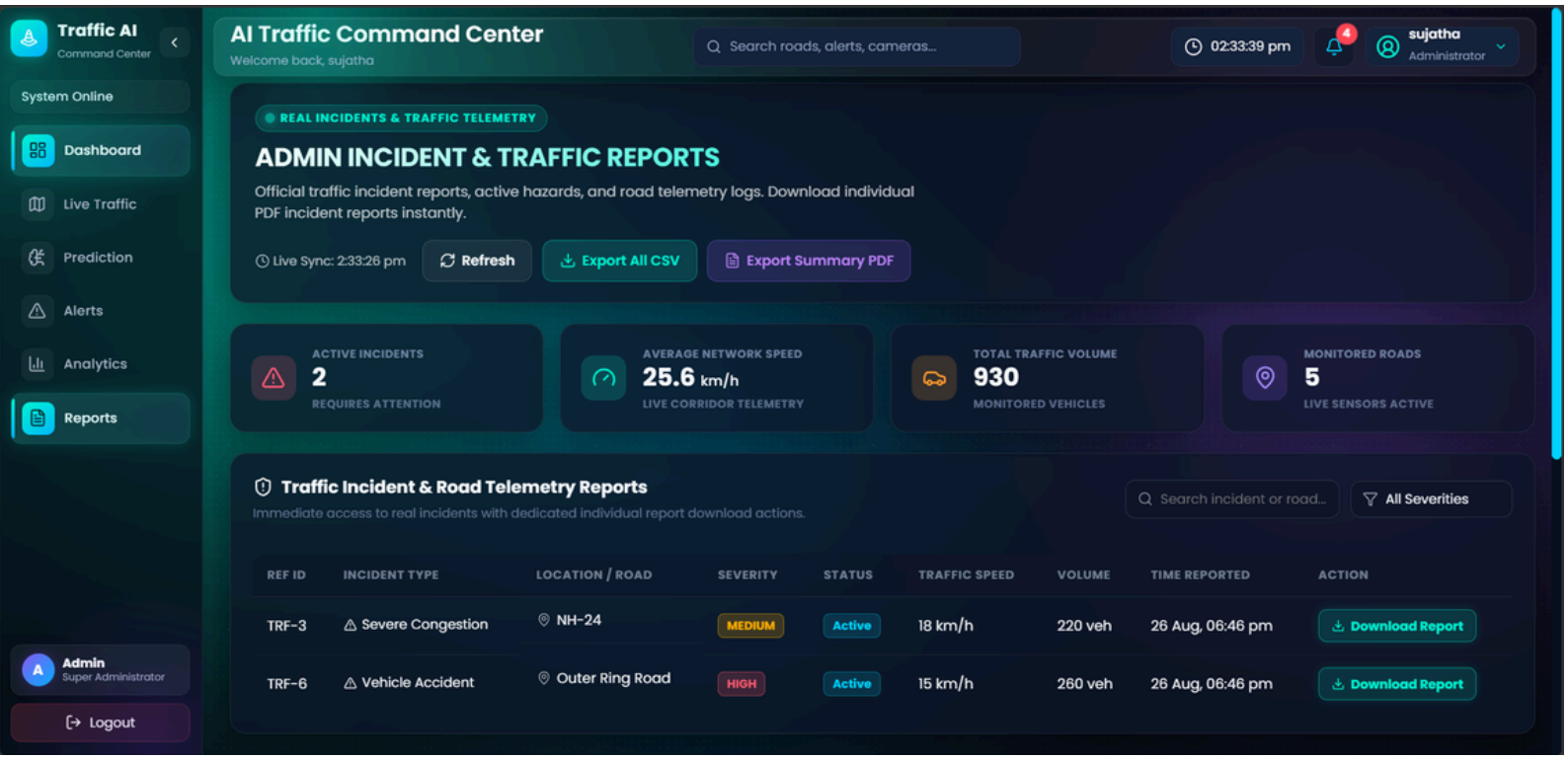


Analytics: Provides traffic trends, congestion heatmaps, road performance, vehicle density, average speed, and historical traffic insights.



Traffic Alerts: Shows active traffic incidents categorized by severity and status, with options to view, acknowledge, resolve, edit, or delete alerts.





Reports: Displays incident and road telemetry reports with traffic speed, vehicle volume, severity, and downloadable CSV/PDF report options.

TRAFFICVISION AI - INCIDENT REPORT

Ref ID: TRF-3 | Generated: 10/9/2026, 2:34:02 pm

1. Incident Details & Status

Field	Detail
Incident Reference	TRF-3
Incident Type	Severe Congestion
Location / Road	NH-24
Severity Level	MEDIUM
Current Status	ACTIVE
Reported Time	26/8/2026, 6:46:59 pm
Description	High vehicle volume (220 vehicles) on NH-24.

2. Associated Live Traffic Telemetry

Telemetry Metric	Recorded Value
Road Name	NH-24
Average Speed	18 km/h
Vehicle Count	220 vehicles
Congestion Level	High
Weather Condition	Clear

11) GITHUB & VERSION CONTROL

11.1 REPOSITORY

GitHub Repository:
springboardmentor01092s-tech/Team_3_Smart_Traffic_AI

Main Branch: main

11.2 REPOSITORY STRUCTURE

```
Team_3_Smart_Traffic_AI/
│
├── Backend/
│   ├── app/
│   │   ├── routers/
│   │   │   ├── auth.py
│   │   │   ├── traffic.py
│   │   │   ├── prediction.py
│   │   │   ├── routes.py
│   │   │   ├── reports.py
│   │   │   ├── alerts.py
│   │   │   └── analytics.py
│   │   ├── tests/
│   │   ├── main.py
│   │   ├── models.py
│   │   ├── schemas.py
│   │   └── database.py
│   ├── Dockerfile
│   └── pytest.ini
│
├── frontend/
│   ├── src/
│   │   ├── pages/
│   │   ├── components/
│   │   ├── styles/
│   │   └── setupTests.js
│   ├── Dockerfile
│   └── nginx.conf
│
├── docker-compose.yml
├── DEPLOYMENT.md
└── TESTING.md
```

The Backend contains the FastAPI application, API routers, database models, schemas, authentication, AI prediction and testing modules. The Frontend contains the user interface, pages, reusable components and styling. Docker configuration files are included to support deployment and environment setup.

11.3 Version Control:

Git and GitHub are used to manage the project's source code and coordinate development among team members.

- Branches: Each team member works on an individual branch to develop features independently. The project also uses the review branch for integrating and reviewing completed changes before they are considered stable.
- Commits: Changes are recorded using Git commits with descriptive messages, allowing the team to track the development history and identify specific feature implementations.
- Pull Requests: Completed feature branches can be merged into the shared integration branch after reviewing the changes, helping prevent conflicts and maintain code quality.
- Issue Tracking: GitHub Issues can be used to record bugs, feature requirements, testing tasks and development problems, allowing the team to track pending work and improvements.
- Remote Repository: Changes are pushed to the team's GitHub repository so that all team members can access the latest shared code.

12) ERROR HANDLING & SECURITY

12.1 ERROR HANDLING

•**Invalid input**: Request payloads are validated at the API layer (schema/type validation) before reaching business logic, returning structured 4xx error responses with clear messages..
•**Database errors:** Database operations are wrapped in try/except blocks with transaction rollback on failure, and return generic, non-leaking error messages to the client while logging full details server-side.
•**AI/ML Errors:** Invalid or incomplete prediction inputs are validated before processing. Errors during model loading or prediction are handled by the backend and an appropriate error response is returned.
•**Authentication errors:** Invalid credentials, expired tokens, and malformed JWTs return clear 401 responses without revealing whether the failure was due to a wrong username or password.

12.2 SECURITY

• **Authentication:** JWT-based stateless authentication issued at login and validated on every protected request.
• **Authorization:** Role-based access control ensures authority/operator-only endpoints (e.g., issuing alerts, managing roads) are inaccessible to public accounts.
• **Password protection:** Passwords are hashed (e.g., bcrypt) before storage; plaintext passwords are never persisted or logged.
• **Sensitive data protection:** Environment-specific secrets (API keys, database credentials) are kept in environment variables rather than source code, and sensitive fields are excluded from logs and error responses.

13)TESTING

- Backend Testing: 18 backend tests were implemented using Pytest to verify authentication, traffic, alerts, and backend workflows.
- Frontend Testing: 5 frontend tests were implemented using Jest to validate important frontend components and functionality.
- AI/ML Testing: The Random Forest model was evaluated using the test dataset to verify congestion classification.
- Integration Testing: Frontend and backend integration was tested to verify API communication, authentication, prediction, reports, alerts, and other application workflows.
- End-to-End Testing: Major user workflows were manually tested to identify and resolve functional, navigation, and access-control issues.

14) RESULTS

14.1 AI/ML Results

METRIC	VALUE
Accuracy	94.29%
Precision (macro avg)	0.94
Recall(macro avg)	0.94
F1 Score(macro avg)	0.94

14.2 System Results

• **API Response Time:** Key API endpoints showed response times ranging from approximately 1–26 ms, excluding authentication operations.
• **AI/ML Inference Time:** Mean inference time was 9.93 ms, with a minimum of 8.17 ms, maximum of 23.67 ms, and P95 of 15.96 ms.

Overall Functionality
• Traffic monitoring, AI-based congestion prediction, route recommendation, alerts, analytics, and reports were tested and verified successfully.
• The Random Forest model successfully classified traffic into Low, Medium, and High congestion levels and returned confidence values.
• Frontend–backend integration was verified across the major application workflows.

15) DEPLOYMENT

TrafficVision AI is prepared for containerized deployment using Docker and Docker Compose. Separate Docker configurations are provided for the backend and frontend components, allowing the application to be deployed as coordinated services.

The FastAPI backend is executed using Uvicorn, while the frontend deployment configuration uses Nginx to serve the React application. Docker Compose is used to manage the application services and simplify the deployment process.

The project also includes deployment documentation describing the required setup and execution steps. The application can be run locally during development using the FastAPI development server and React development environment, while the Docker configuration provides a consistent environment for deployment.

Git and GitHub are used for version control, branch management, collaboration, and maintaining the project source code.

16) CHALLENGES & LIMITATIONS

1. Traffic Data Availability

Obtaining consistent and realistic traffic data for different roads and traffic conditions was challenging. The system therefore works with structured traffic data for development and testing.

2. AI Model Integration

Integrating the trained Random Forest model with the FastAPI backend required maintaining the same feature selection and encoding during both training and prediction.

3. Frontend–Backend Integration

Connecting the React frontend with multiple FastAPI APIs required careful handling of requests, responses, authentication, and role-based access.

4. Map and System Integration

Integrating traffic information with the interactive map, prediction results, alerts, reports, and analytics required coordination between multiple modules of the application

17) FUTURE SCOPE

•Integrate with live, real-world traffic sensors and CCTV feeds to replace/augment sample data, improving both monitoring accuracy and prediction reliability.

•Add resilient multi-provider fallback (beyond simple caching) for mapping/traffic APIs to reduce dependence on any single external service.

•Retrain and continuously validate prediction models on live data as it accumulates, to improve generalization across roads, seasons, and events.

•Pilot the platform on a subset of real city roads with hardware partners to validate performance under real sensor and network conditions.

•Extend the platform to a native mobile app for commuters, and scale the architecture (via Kubernetes) for multi-city / citywide deployment.

•Add deeper analytics such as event-based congestion prediction (e.g., matches, concerts, weather emergencies) and integration with emergency-response systems.

18) CONCLUSION

TrafficVision AI addresses reactive traffic management by combining traffic monitoring with AI-based congestion prediction, route recommendation, alerts, analytics, and reports through a role-based platform. The current implementation uses structured traffic data and provides a foundation for future integration with real-world traffic sources and larger-scale deployment.

THANK YOU

~TEAM 3